

Matrix Block Sum:-

Code:-

```
class Solution {
    public int[][] matrixBlockSum(int[][] mat, int k) {
        int m = mat.length;
        int n = mat[0].length;
        int[][] pref = new int[m + 1][n + 1];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                pref[i + 1][j + 1] = mat[i][j]
                    + pref[i][j + 1]
                    + pref[i + 1][j]
                    - pref[i][j];
            }
        }
        int[][] ans = new int[m][n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                int r1 = Math.max(0, i - k);
                int c1 = Math.max(0, j - k);
                int r2 = Math.min(m - 1, i + k);
                int c2 = Math.min(n - 1, j + k);
                ans[i][j] = pref[r2 + 1][c2 + 1]
                    - pref[r1][c2 + 1]
                    - pref[r2 + 1][c1]
                    + pref[r1][c1];
            }
        }
        return ans;
    }
}
```

Output:-

The screenshot displays the LeetCode interface for the 'Matrix Block Sum' problem. The problem is marked as 'Accepted' with 12/12 testcases passed. The submission was made by 'tameem_shaikh' on September 18, 2026, at 11:35. The performance metrics show a runtime of 7 ms (beats 60.24%) and a memory usage of 46.73 MB (beats 41.04%). The code is written in Java and is shown in the 'Code' tab. The 'Testcase' tab shows the input and output for the problem.

Input:

```
mat =
[[1,2,3], [4,5,6], [7,8,9]]

k =
1
```

Output:

```
[[12,21,16], [27,45,33], [24,39,28]]
```

Expected:

```
[[12,21,16], [27,45,33], [24,39,28]]
```

Matrix Layer Rotation:-

Code:-

```
import java.io.*;
import java.util.*;
import java.util.stream.*;
import static java.util.stream.Collectors.toList;

class Result {

    public static void matrixRotation(List<List<Integer>> matrix, int r) {
        int m = matrix.size();
        int n = matrix.get(0).size();

        int layers = Math.min(m, n) / 2;

        for (int layer = 0; layer < layers; layer++) {

            int top = layer;
            int bottom = m - 1 - layer;
            int left = layer;
            int right = n - 1 - layer;

            List<Integer> list = new ArrayList<>();

            for (int j = left; j <= right; j++) {
                list.add(matrix.get(top).get(j));
            }

            for (int i = top + 1; i <= bottom; i++) {
                list.add(matrix.get(i).get(right));
            }

            for (int j = right - 1; j >= left; j--) {
                list.add(matrix.get(bottom).get(j));
            }

            for (int i = bottom - 1; i > top; i--) {
                list.add(matrix.get(i).get(left));
            }

            int len = list.size();
            int rotate = r % len;

            int index = 0;

            for (int j = left; j <= right; j++) {
                matrix.get(top).set(j, list.get((index + rotate) % len));
                index++;
            }

            for (int i = top + 1; i <= bottom; i++) {
```

```

        matrix.get(i).set(right, list.get((index + rotate) % len));
        index++;
    }

    for (int j = right - 1; j >= left; j--) {
        matrix.get(bottom).set(j, list.get((index + rotate) % len));
        index++;
    }

    for (int i = bottom - 1; i > top; i--) {
        matrix.get(i).set(left, list.get((index + rotate) % len));
        index++;
    }
}

for (List<Integer> row : matrix) {
    for (int value : row) {
        System.out.print(value + " ");
    }
    System.out.println();
}
}
}

```

```

public class Solution {

```

```

    public static void main(String[] args) throws IOException {

```

```

        BufferedReader bufferedReader =
            new BufferedReader(new InputStreamReader(System.in));

```

```

        String[] firstMultipleInput =
            bufferedReader.readLine()
                .replaceAll("\\s+$", "")
                .split(" ");

```

```

        int m = Integer.parseInt(firstMultipleInput[0]);
        int n = Integer.parseInt(firstMultipleInput[1]);
        int r = Integer.parseInt(firstMultipleInput[2]);

```

```

        List<List<Integer>> matrix = new ArrayList<>();

```

```

        IntStream.range(0, m).forEach(i -> {
            try {
                matrix.add(
                    Stream.of(
                        bufferedReader.readLine()
                            .replaceAll("\\s+$", "")
                            .split(" ")
                    )
                    .map(Integer::parseInt)

```

```

        .collect(toList())
    );
} catch (IOException ex) {
    throw new RuntimeException(ex);
}
});

Result.matrixRotation(matrix, r);

bufferedReader.close();
}
}

```

Output:-

✓ Test case 0

✓ Test case 1

✓ Test case 2

✓ Test case 3

✓ Test case 4

✓ Test case 5

✓ Test case 6

Compiler Message

Success

Input (stdin)

Download

1	4	4	1
2	1	2	3 4
3	5	6	7 8
4	9	10	11 12
5	13	14	15 16

Expected Output

Download

1	2	3	4	8
---	---	---	---	---